Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea practică Nr. 6

Disciplina: Tehnici de programare

Tema: ” Informații invizibile: forensics și steganografie ”

A efectuat : st. gr. SI – 263 Berejnoi Alexandr
A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Realizarea, în Python, a două unelte de care intră în arsenalul de securitate:

- **Lucrarea A** – decoder universal (`arsenal/forensics/decode.py`): recunoaște și desface straturile de codare (base64, hex, URL), sparge un XOR cu cheie de un octet și hash-uri de parole prin dicționar;
- **Lucrarea B** – forensics pe fișiere: tipul real după magic bytes, textul și metadatele (EXIF, GPS) dintr-o imagine, extragerea unui fișier ascuns (file carving) și steganografie LSB.

Toate analizele au fost efectuate exclusiv pe fișierele de laborator, generate local cu scriptul `probe/pregateste.py`.

DESFĂȘURAREA LUCRĂRII

1. Pregătirea mediului

În mediul virtual activat `(.venv)` a fost instalată biblioteca Pillow; numele pachetului la instalare este „pillow”.

```
(.venv) tbm@Panoptic L6 % pip install pillow
Collecting pillow
  Using cached pillow-13.0.0-cp314-cp314-macosx_11_0_arm64.whl.metadata (9.1 kB)
Using cached pillow-13.0.0-cp314-cp314-macosx_11_0_arm64.whl (4.9 MB)
Installing collected packages: pillow
Successfully installed pillow-13.0.0
```

Figura 1.1 - Instalarea bibliotecii Pillow

Apoi a fost rulat scriptul de pregătire, care creează în folderul `probe/` țintele analizate în lucrare: șirul codat în straturi, mesajul XOR, hash-urile cu dicționarul, poza cu EXIF și GPS, poza cu arhivă ascunsă și fișierele pentru vârful sălii.



Figura 1.2 - Generarea țintelor de laborator în folderul probe/

2. Lucrarea A. Decoderul universal

A1. Text și octeți

Diferența dintre text (`str`) și octeți (`bytes`) stă la baza sarcinii: `b[0]` este un număr, iar `hex()` și `bytes.fromhex()` convertesc. Funcția `salut()` din `decode.py` arată acest lucru.



Figura 2.1 - Text, octeți și hex (funcția salut)

A2. Recunoașterea stratului după formă

Funcția `ghici_strat` clasifică șirul doar după caractere, fără să-l decodeze. Ordinea verificărilor contează: întâi URL (`%xx`), apoi hex, la urmă base64 (cel mai larg alfabet), deoarece un șir hex este și un șir base64 valid. (python -c "from arsenal.forensics.decode import ghici_strat as g; [print(repr(s), '->', g(s)) for s in ['%2Fetc', '48656c6c6f', 'U2FsdXQ=', 'Salut!']]")



Figura 2.2 - Recunoașterea stratului (ghici_strat)

A3. Desfacerea unui strat

Funcția `desfa` decodează stratul recunoscut și întoarce perechea (strat, rezultat în octeți). La base64 s-a folosit `validate=True`, astfel încât un șir care doar seamănă a base64 să nu fie decodat în tăcere; erorile de decodare sunt prinse și tratate ca „necunoscut”. (python -c "from arsenal.forensics.decode import desfa; print(desfa('U2FsdXQ=')); print(desfa('48656c6c6f')); print(desfa('%2Fetc'))")



Figura 2.3 - Desfacerea unui singur strat (desfa)

A4. Straturi multiple

Un rezultat poate fi el însuși o codare. Dacă bucla s-ar opri la primul rezultat „citibil”, `mesaj.txt` s-ar opri după un singur pas, deoarece și textul base64, și textul hex sunt imprimabile. De aceea funcția `desfa_straturi` acceptă un strat doar dacă rezultatul lui este text citibil; altfel, un cuvânt obișnuit ca `password` (alfabet valid base64) ar fi „decodat” în gunoi. Parametrul `max_pasi` protejează împotriva buclei infinite.



Figura 2.4 - Mesajul din mesaj.txt, desfăcut strat cu strat

Mesajul ascundea textul `FLAG{straturi_de_encoding}` sub trei straturi: hex, peste care s-au aplicat două straturi de base64. Pentru exemplul din fișa de laborator (`NTM1OTU3NTk=`) rezultatul este `SYWY`, după straturile base64 și hex; următorul pas ar da octeți ilizibili, deci bucla se oprește corect.



Figura 2.5 - Exemplul din fișa de laborator

A5. XOR cu cheie de un octet

O cheie de un octet are doar 256 de valori, deci au fost încercate toate (bruteforce). Filtrul „toți octeții sunt imprimabili” nu este însă suficient: în fișierul analizat **17 chei** produc octeți imprimabili, deoarece unele chei schimbă doar câțiva biți. Candidații au fost ordonați după ponderea literelor și a spațiilor (`scor_text`). O primă variantă număra și cifrele și a ales o cheie greșită, fiindcă cu cheia greșită spațiile devin cifre; scorul a fost restrâns la litere și spații.



Figura 2.6 - Cheia XOR găsită prin forță brută

A6. Hash-uri sparte prin dicționar

Lungimea hash-ului indică algoritmul: 32 de caractere – MD5, 40 – SHA-1, 64 – SHA-256. Un hash nu se „decriptează”: fiecare cuvânt din dicționar este hașurat și comparat cu ținta. Au fost sparte toate cele cinci hash-uri din `hashuri.txt` (toate MD5): `password`, `dragon`, `qwerty`, `letmein`, `monkey`.



Figura 2.7 - Parolele sparte prin dicționar

A7. argparse, fișa JSON și testul

Unealta este comandată din linia de comandă: intrarea este un șir sau (cu `--fisier`) o cale, iar `--xor` și `--hash WORDLIST` aleg modul de spargere. Rezultatul se salvează în `fisa.json`, lângă script (cale calculată cu `Path(__file__)`, ca să nu depindă de folderul curent). Fișa conține rezultatul ultimei rulări.



Figura 2.8 - Conținutul fișierului fisa.json

Testele din `teste/test_decode.py` acoperă recunoașterea stratului, desfacerea base64/hex/URL, straturile multiple, cuvântul obișnuit care nu trebuie decodat, XOR-ul și hash-ul. Ele se rulează din rădăcina depozitului (pachetele `arsenal/` și `arsenal/forensics/` conțin câte un `__init__.py` gol, pentru ca importul să funcționeze).

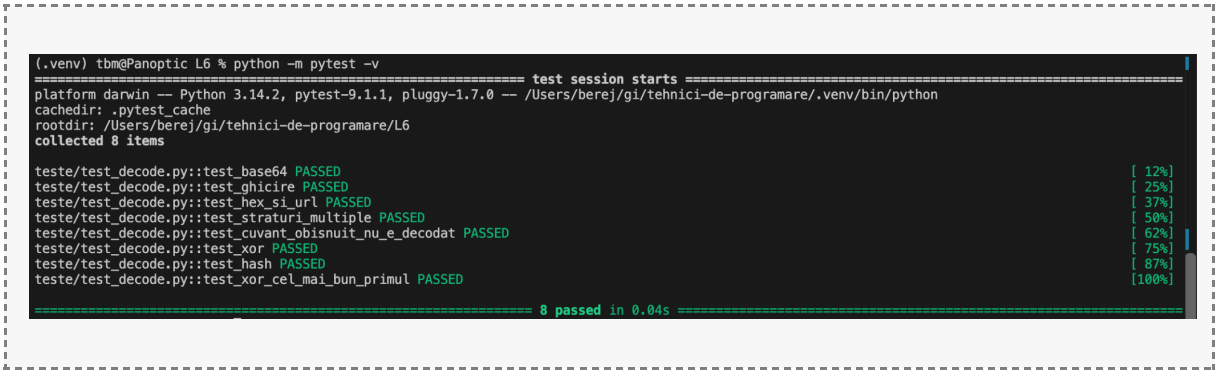


Figura 2.9 - Rezultatul rulării testelor

3. Lucrarea B. Forensics pe fișiere și un mesaj ascuns

Pașii B1–B4 sunt rulați cu un singur script, `forensics.py`, care afișează câte o secțiune pentru fiecare pas.

B1. Magic bytes: tipul real, nu extensia

Tipul unui fișier este dat de primii lui octeți, nu de extensie. Fișierele se deschid obligatoriu în modul binar (`"rb"`), altfel octeții se pot altera. Pentru fișierele fără semnătură cunoscută se întorc primii octeți în hex, iar dacă toți sunt caractere imprimabile, fișierul este etichetat „text”. `foto.jpg` este JPEG,

`ascuns.png` este PNG, `mesaj.txt` este text, iar `xor.bin` nu are semnătură cunoscută (octeți cu bitul superior setat, ceea ce se potrivește cu un text trecut prin XOR cu `0x80`).

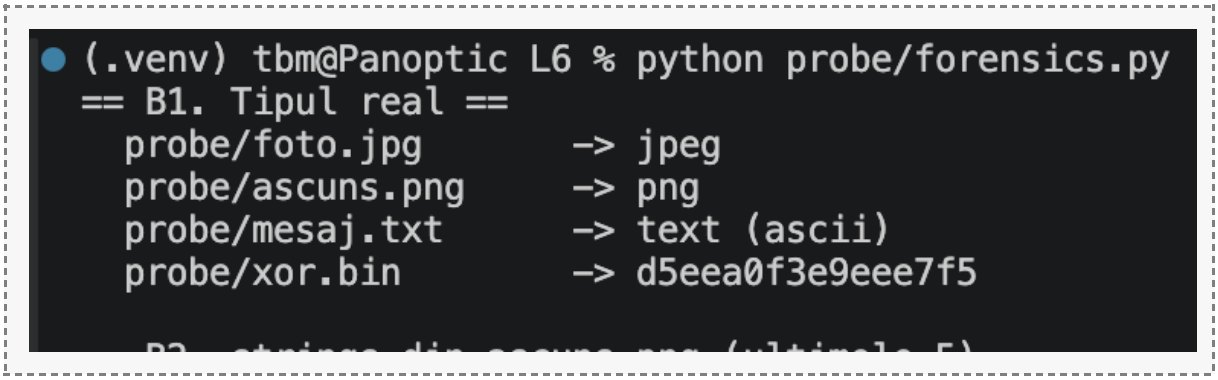


Figura 3.1 - Tipul real al fișierelor (tip_real)

B2. strings: textul citibil dintr-un binar

Secvențele de caractere imprimabile (minimum 4) se scot cu o expresie regulată aplicată pe octeți, ca în utilitarul `strings`. După marcajul `IEND` (sfârșitul real al PNG-ului) apar numele `secret.txt`, un text de tip `FLAG{...}` și `PK` – semnătura unei arhive ZIP. Este un indiciu clar că imaginea ascunde un fișier.

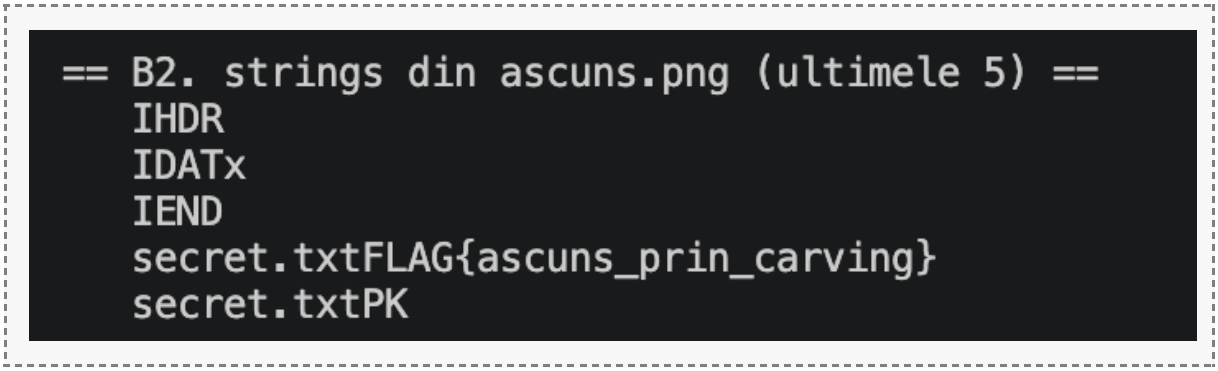


Figura 3.2 - Șirurile imprimabile din ascuns.png (strings)

B3. Metadate EXIF și GPS

Blocul EXIF s-a citit cu `getexif()`, iar sub-blocul GPS (tag 34853, adică `0x8825`) cu `get_ifd()`; metoda `_getexif()`. Coordonatele vin ca (grade, minute, secunde) plus emisferă și se convertesc în grade zecimale: grade + minute/60 + secunde/3600, cu semn minus pentru emisferele S și W. Poza a fost făcută cu un Apple iPhone 13, la 14.09.2026, ora 10:30, la 47°1′0″ N, 28°51′27″ E, adică 47.016667; 28.857500. Acesta este motivul pentru care metadatele

EXIF se șterg înainte de publicarea unei fotografii: ele dau locul și momentul exact al fotografierii.



Figura 3.3 - Metadatele EXIF și coordonatele GPS din foto.jpg

B4. File carving: fișierul ascuns

Semnătura ZIP (`PK\x03\x04`) a fost căutată abia după chunk-ul `IEND` al PNG-ului, astfel încât o apariție întâmplătoare a acestor octeți în datele imaginii să nu producă o potrivire falsă. De la poziția găsită, octeții au fost salvați în `gasit.zip` și deschiși cu `zipfile`: arhiva începe la octetul 781 și conține `secret.txt` cu textul `FLAG{ascuns_prin_carving}`.

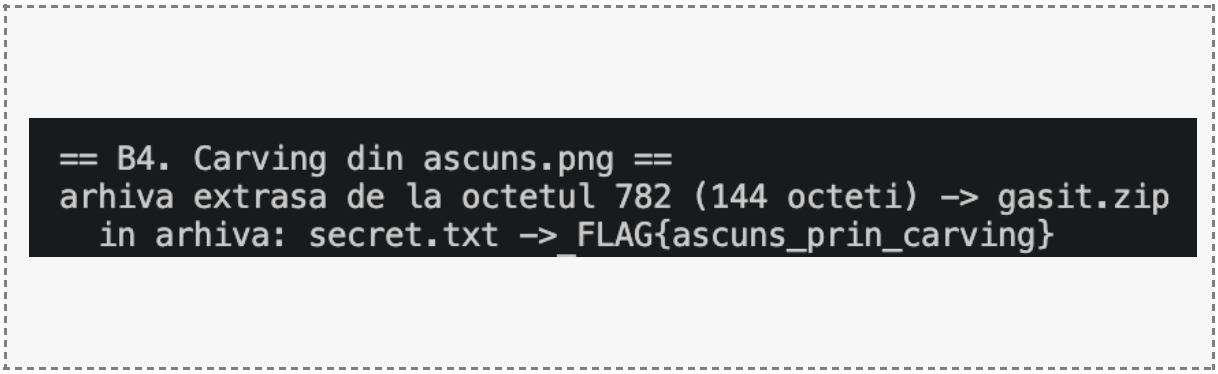


Figura 3.4 - Arhiva extrasă din ascuns.png (carving)

B5. Ascunderea unui mesaj prin LSB

Pixelii imaginii gazdă (`foto.jpg`) sunt priviți ca un șir plat de {R, G, B, R, G, B, …}. Fiecare bit al mesajului înlocuiește bitul cel mai puțin semnificativ al unui canal: `canal = (canal & ~1) | bit`. La mesaj se adaugă un octet `0` ca terminator. Rezultatul se salvează în PNG (fără pierdere), deoarece un JPG ar recomprima imaginea și ar distruge biții ascunși. Capacitatea imaginii de 640 × 480 pixeli este 640 · 480 · 3 = 921 600 de biți (115 200 bytes).

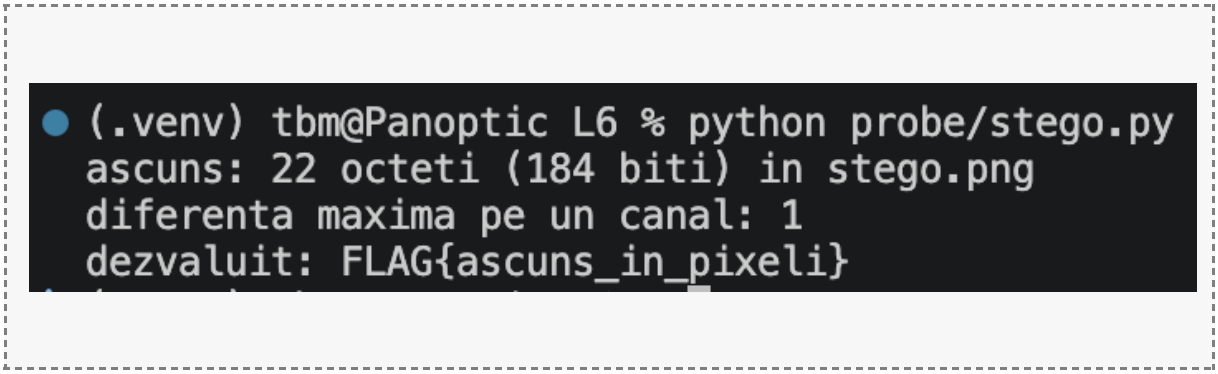


Figura 3.5 - Ascunderea mesajului în pixeli (ascunde)

B6. Dezvăluirea mesajului

Dezvăluirea este inversul ascunderii: se adună ultimii biți ai canalelor, se grupează câte 8 în octeți și se oprește citirea la primul octet 0. Diferența maximă dintre imaginea originală și `stego.png` este 1 pe canal, deci modificarea este invizibilă ochiului, iar mesajul `FLAG{ascuns_in_pixeli}` a fost recuperat integral.

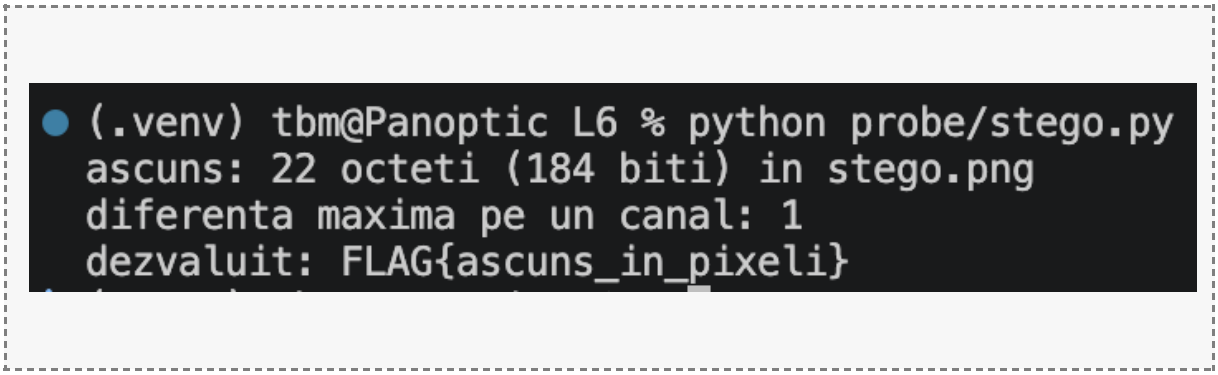


Figura 3.6 - Mesajul LSB dezvăluit din stego.png (dezvaluie)

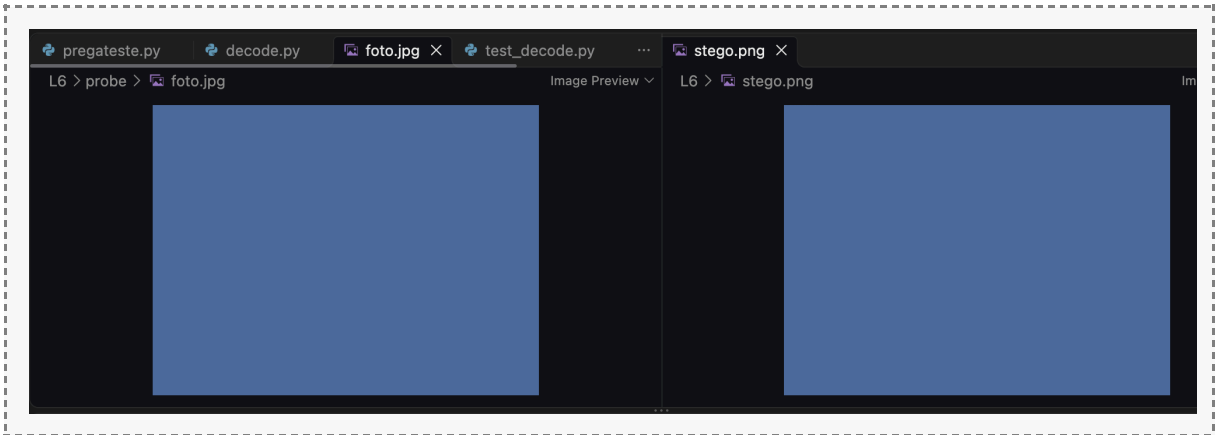


Figura 3.7 - Imaginea originală și imaginea cu mesaj ascuns

4. Vârful: challenge.bin

Fișierul combină tot ce s-a făcut în lucrare: un strat de base64 peste un XOR cu cheie de un byte, cu un hash MD5 de verificare în `challenge_hint.txt`. Pașii: (1) se scoate stratul base64, obținând octeți ilizibili; (2) se încearcă toate cele 256 de chei XOR; (3) fiecare candidat se verifică cu MD5-ul din indiciu. Verificarea cu hash este obligatorie: scorul de text singur nu alege aici cheia corectă, iar 12 candidați imprimabili s-au dovedit falși. Cheia potrivită este `0x80`, iar textul clar este `FLAG{lant_complet_despicat}`.



Figura 4.1 - Lanțul complet despicat (challenge.bin)

Rezultate

Sarcina	Rezultat
Mesaj cu straturi (mesaj.txt)	base64 → base64 → hex; `FLAG{straturi_de_encoding}`
XOR (xor.bin)	cheia `0x80` (128); `FLAG{xor_un_octet}`
Hash-uri (hashuri.txt, MD5)	password, dragon, qwerty, letmein, monkey
Tipul real al fișierelor	foto.jpg – jpeg; ascuns.png – png; mesaj.txt – text; xor.bin – fără semnătură cunoscută
Metadate EXIF	Apple iPhone 13; 2026:09:14 10:30:00
GPS	47.016667, 28.857500
Arhiva ascunsă în ascuns.png	ZIP la octetul 781; `secret.txt` cu `FLAG{ascuns_prin_carving}`
Mesaj LSB	`FLAG{ascuns_in_pixeli}` (stego.png)
challenge.bin	base64 peste XOR cu cheia `0x80`; `FLAG{lant_complet_despicat}`

CONCLUZII

În această lucrare am creat două unelte în Python. Prima este un decoder care desface șiruri codate în mai multe straturi (base64, hex, URL), sparge un XOR cu cheie de un octet și află parole din hash-uri cu ajutorul unui dicționar. A doua este un set de funcții pentru analiza fișierelor: aflarea tipului real, citirea textului, a metadatelor EXIF și a coordonatelor GPS, extragerea unei arhive ascunse într-o imagine, precum și ascunderea și dezvăluirea unui mesaj prin steganografie LSB. Rezultatele au fost obținute pe fișiere generate local, iar testele decoderului trec.

Un XOR cu cheie de un byte se sparge în cel mult 256 de încercări. Hash-ul nu se inversează, dar parolele slabe se găsesc prin dicționar. Extensia unui fișier nu arată ce conține el, de aceea se verifică primii octeți. O fotografie poate da locul și ora la care a fost făcută, deci metadatele trebuie șterse înainte de publicare. Pe parcurs au apărut și câteva probleme, rezolvate în cod: la straturile multiple nu m-am oprit la primul text citibil, la XOR mai multe chei dădeau text imprimabil, iar la carving am căutat arhiva abia după sfârșitul imaginii. Uneltele s-au folosit doar pe fișierele de laborator, fiindcă analiza fișierelor altei persoane fără acord este acces neautorizat la date.

BIBLIOGRAFIE

1. https://metanit.com/python/
2. https://pillow.readthedocs.io/en/stable/
3. Fișa de laborator „Laboratorul 6. Informații invizibile: forensics și steganografie”, Tehnici de programare, UTM, FCIM.